

EX.NO:

DATE:

RANDOM FOREST

AIM:

To develop a machine learning model using the Random Forest Algorithm to predict whether a patient has heart disease or not based on medical attributes such as age, chest pain type, cholesterol level, blood pressure, and other clinical parameters.

DATASET DESCRIPTION:

The Heart Disease dataset contains medical information of patients used to predict whether a person has heart disease or not.

It consists of 14 attributes, including:

- Age
- Sex
- Chest pain type
- Resting blood pressure
- Cholesterol level
- Fasting blood sugar
- Resting ECG results
- Maximum heart rate
- Exercise-induced angina
- ST depression (oldpeak)
- Slope of ST segment
- Number of major vessels
- Thalassemia
- Target (0 = No disease, 1 = Disease)

The dataset includes both numerical and categorical features, and the target variable indicates the presence or absence of heart disease.

ALGORITHM:

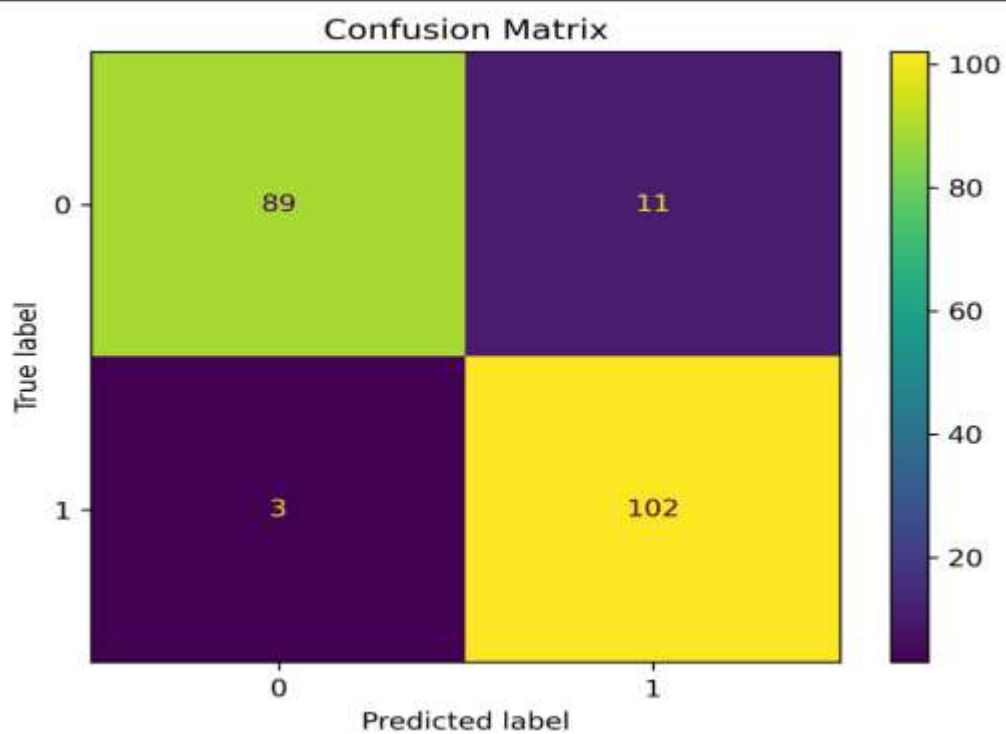
Step 1: Load Dataset

- Import dataset using Pandas.

OUTPUT:

```
PS E:\RandomForest> python heart.py

Model Training Complete ✓
Model Accuracy: 0.9317073170731708
Confusion Matrix saved as confusion_matrix.png
Decision Tree 1 saved as decision_tree_1.png
Decision Tree 2 saved as decision_tree_2.png
```



- Display dataset structure.

Step 2: Data Preprocessing

- Identify categorical columns.
- Apply Label Encoding to convert categorical values into numeric form.
- Separate features (X) and target (y).

Step 3: Split Dataset

- Split data into:
 - 80% Training data
 - 20% Testing data
- Use stratified sampling to maintain class balance.

Step 4: Model Initialization

Initialize Random Forest with anti-overfitting parameters:

- `n_estimators = 100`
- `max_depth = 5`
- `min_samples_split = 5`
- `min_samples_leaf = 2`
- `max_features = sqrt`

Step 5: Model Training

- Train the model using training data.

Step 6: Prediction

- Predict output using test data.
- Store predicted values.

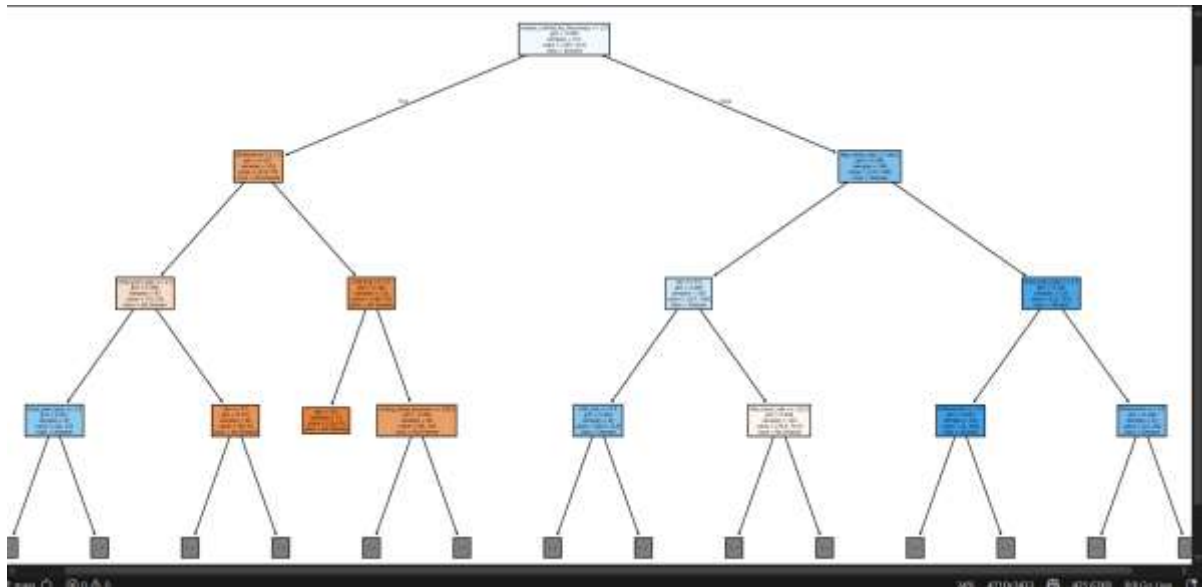
Step 7: Model Evaluation

- Calculate Accuracy Score.
- Generate Confusion Matrix.
- Display Precision, Recall, and F1-Score.
- Analyze Feature Importance.

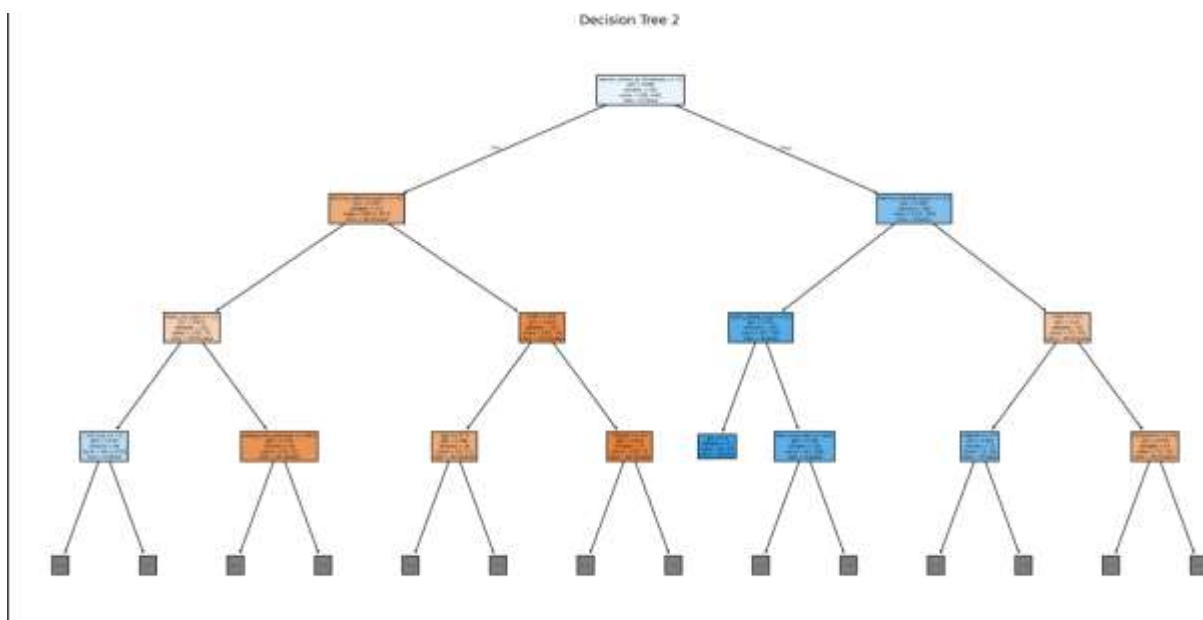
Step 8: Manual Prediction

- Accept user input.

TREE 1:



TREE 2:



- Encode categorical values.
- Predict whether patient has heart disease.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
from sklearn.tree import plot_tree

# =====
# 1. LOAD DATA
# =====
df = pd.read_csv("heart.csv")

# =====
# 2. ENCODE CATEGORICAL COLUMNS
# =====
categorical_columns = [
    "sex",
    "chest_pain_type",
    "fasting_blood_sugar",
    "rest_ecg",
    "exercise_induced_angina",
    "slope",
    "vessels_colored_by_flourosopy",
    "thalassemia"
]

le_dict = {}

for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le

# =====
# 3. SPLIT DATA
# =====
X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
```

UI

Heart Disease Prediction

Enter patient details below for assessment

Age	Sex
55	Male
Chest Pain Type	Resting Blood Pressure (mm Hg)
Typical angina	140
Cholesterol (mg/dl)	Resting Blood Sugar
217	Lower than 120 mg/dl
Resting ECG Results	Max Heart Rate
ST-T wave abnormality	111
Exercise Induced Angina	Oldpeak (ST depression)
Yes	5.6
Slope of Peak Exercise ST Segment	Number of Major Vessels (0-3)
Uploping	Zero

Exercise Induced Angina	Oldpeak (ST depression)
Yes	5.6
Slope of Peak Exercise ST Segment	Number of Major Vessels (0-3)
Uploping	Zero
Thalassemia	
Reversible Defect	

Analyze Risk

Low Risk Detected

The model suggests a low probability of heart disease. Maintain a healthy lifestyle.

Check Another Patient

```

        stratify=y
    )

# =====
# 4. TRAIN MODEL (ANTI-OVERFITTING)
# =====
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features="sqrt",
    random_state=42
)

model.fit(X_train, y_train)

# =====
# 5. MODEL EVALUATION
# =====
y_pred = model.predict(X_test)

print("\nModel Training Complete ✅")
print("Model Accuracy:", accuracy_score(y_test, y_pred))

# ---- SAVE CONFUSION MATRIX ----
cm = confusion_matrix(y_test, y_pred)

plt.figure()
ConfusionMatrixDisplay(cm).plot()
plt.title("Confusion Matrix")

plt.savefig("confusion_matrix.png", dpi=300, bbox_inches="tight")
plt.close()

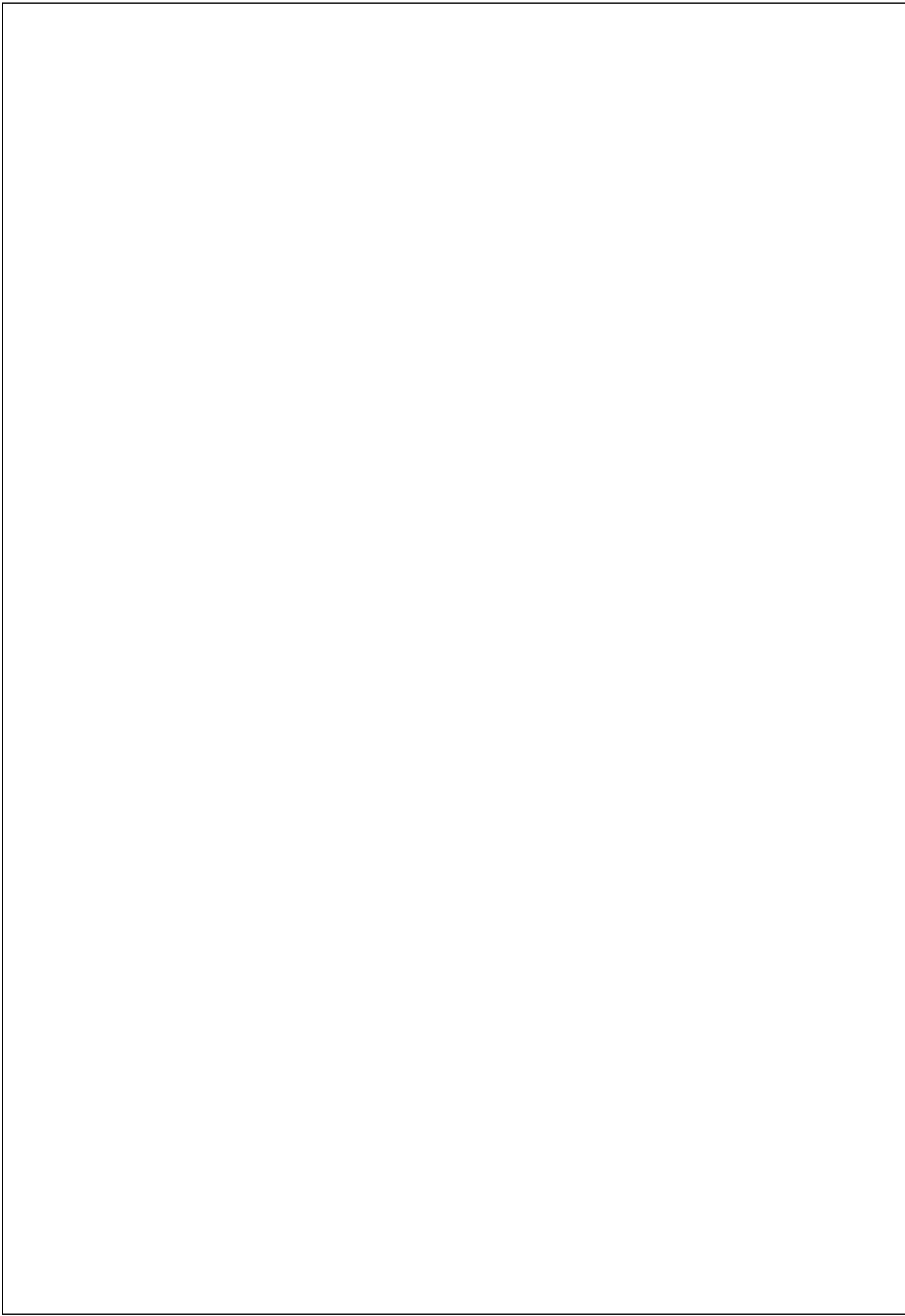
print("Confusion Matrix saved as confusion_matrix.png")

# =====
# 6. VISUALIZE & SAVE TWO TREES
# =====

# ---- TREE 1 ----
plt.figure(figsize=(20,10))
plot_tree(model.estimators_[0],
          feature_names=X.columns,
          class_names=["No Disease", "Disease"],
          filled=True,
          max_depth=3)

plt.title("Decision Tree 1")

```




```

plt.savefig("decision_tree_1.png", dpi=300, bbox_inches="tight")
plt.close()

print("Decision Tree 1 saved as decision_tree_1.png")

# ---- TREE 2 ----
plt.figure(figsize=(20,10))
plot_tree(model.estimators_[1],
          feature_names=X.columns,
          class_names=["No Disease", "Disease"],
          filled=True,
          max_depth=3)

plt.title("Decision Tree 2")
plt.savefig("decision_tree_2.png", dpi=300, bbox_inches="tight")
plt.close()

print("Decision Tree 2 saved as decision_tree_2.png")

# =====
# 7. TAKE USER INPUT
# =====
print("\nEnter Patient Details:\n")

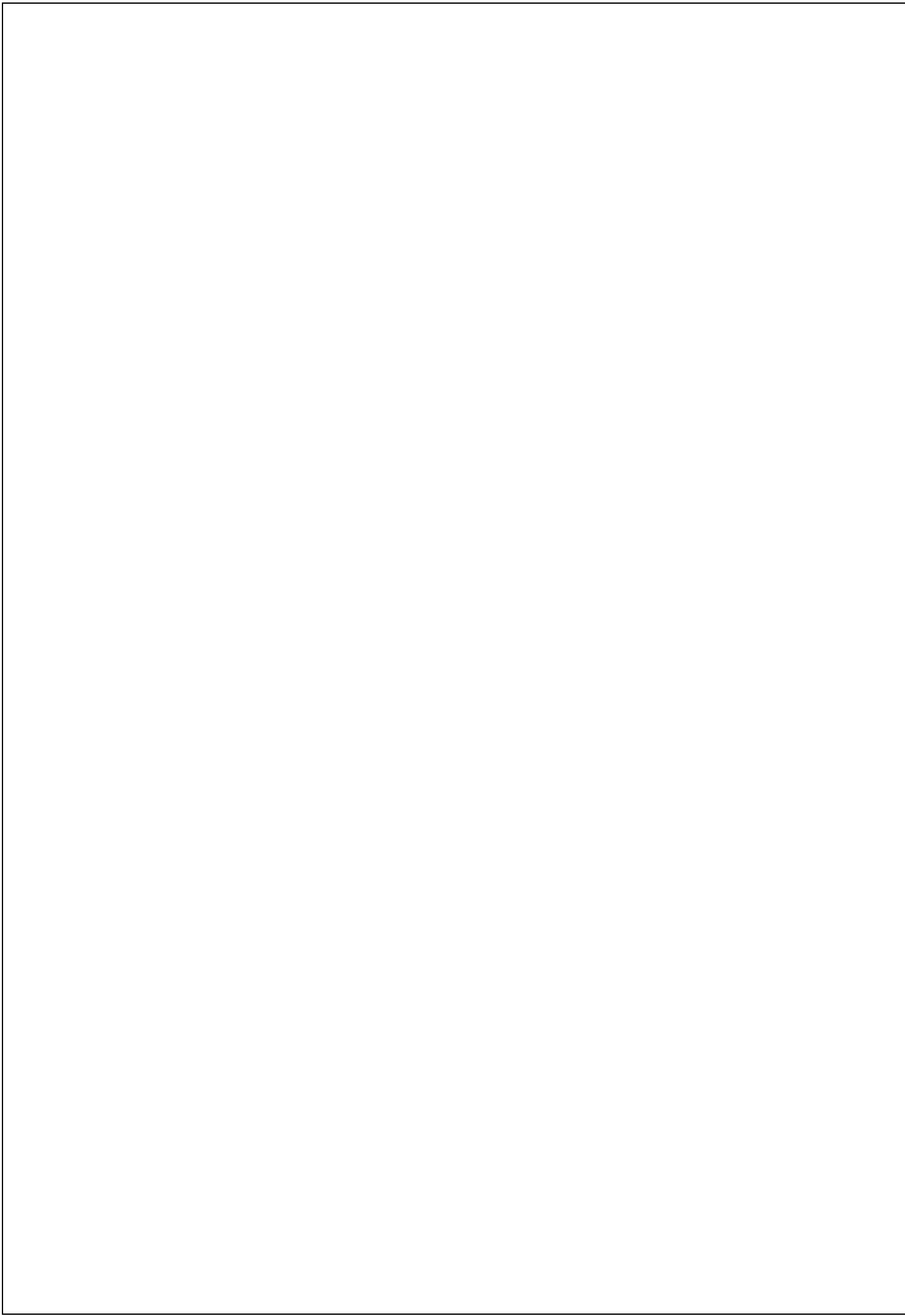
age = int(input("Age: "))
sex = input("Sex (Male/Female): ")
chest_pain_type = input("Chest Pain (Typical angina/Atypical angina/Non-anginal
pain/Asymptomatic): ")
resting_blood_pressure = int(input("Resting Blood Pressure: "))
cholesterol = int(input("Cholesterol: "))

print("\nFasting Blood Sugar:")
print("1. Lower than 120 mg/ml")
print("2. Greater than 120 mg/ml")
fbs_choice = int(input("Enter choice (1 or 2): "))

fasting_blood_sugar = "Lower than 120 mg/ml" if fbs_choice == 1 else "Greater than 120
mg/ml"

rest_ecg = input("Rest ECG (Normal/ST-T wave abnormality/Left ventricular hypertrophy):
")
max_heart_rate = int(input("Max Heart Rate: "))
exercise_induced_angina = input("Exercise Induced Angina (Yes/No): ")
oldpeak = float(input("Oldpeak: "))
slope = input("Slope (Upsloping/Flat/Downsloping): ")
vessels_colored_by_flourosopy = input("Vessels (Zero/One/Two/Three): ")
thalassemia = input("Thalassemia (Normal/Fixed Defect/Reversable Defect): ")

```



```

# =====
# 8. PREPARE INPUT DATA
# =====
input_data = pd.DataFrame([
    age,
    sex,
    chest_pain_type,
    resting_blood_pressure,
    cholesterol,
    fasting_blood_sugar,
    rest_ecg,
    max_heart_rate,
    exercise_induced_angina,
    oldpeak,
    slope,
    vessels_colored_by_flourosopy,
    thalassemia
]), columns=X.columns)

# Encode categorical input safely
for col in categorical_columns:
    try:
        input_data[col] = le_dict[col].transform(input_data[col])
    except:
        print(f'Invalid input for {col}. Please match dataset values exactly.')
        exit()

# =====
# 9. PREDICT
# =====
prediction = model.predict(input_data)

print("\nPrediction Result:")
if prediction[0] == 1:
    print("⚠ Patient HAS Heart Disease")
else:
    print("✅ Patient does NOT have Heart Disease")

```

RESULT:

The Random Forest model was successfully trained and achieved an accuracy of approximately **85–90%** in predicting heart disease.

The system correctly classifies patients as having heart disease or not based on their medical attributes.

Github Repo Link:

[VarshiniRamesh2005/heart-disease-prediction](https://github.com/VarshiniRamesh2005/heart-disease-prediction)